GOOGLE | ALL SOURCES | TOPIC 10 OF 10 | LESS COMMON TOPIC

Grids and Simulation

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	4
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Grids and Simulation | 6 questions | tap a title to jump to it

M	1. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	**1Point3Acres**	**2026-05-18**
M	2. Gate Entry / Exit Time Resolution with Tie-Break Rules	**1Point3Acres**	**2026-05-04**
H	3. Count Overlapping Rectangle Updates on a Grid	**PracHub**	**2026-07-10**
H	4. Simulate In-Place Cellular State Updates	**PracHub**	**2026-05-04**
H	5. Babylon Tile-Stack Merge Game	**1Point3Acres**	**2025-12-29**
H	6. Count same-color squares in a character grid	**PracHub**	**2025-11-22**

STUDY HERE: topic 10, Grids and Simulation

Site tags: object-design, grid, simulation, medium

MEDIUM | 1Point3Acres | phone-screen, tech-screen | 1 of 6

Tic-Tac-Toe System with K Players and 3-in-a-Row Win

ASKED 2026-05-18 | 155 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 18.1** **30 DAYS 0** **PRIORITY 154/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 348 Design Tic-Tac-Toe (related)

Frequency: Single report | Last asked: 2026-05-18 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

QUESTION

Design a Tic-Tac-Toe system for **k** players on an **n x n** board. After each move, return the game status; any player wins immediately after forming 3 consecutive marks in a straight line.

Input parameters: k players and an n x n board.
Players take turns placing their marks on empty cells.
A player wins as soon as they form 3 consecutive marks in a straight line, regardless of how large n is.
After each move, return the correct game status, such as Game Over or <player> won.
Discuss implementation details and time complexity before coding.

APPROACH

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

HOW TO THINK ABOUT IT

A move can only create a win through the cell just played, so after each move look only along the four lines through that cell (row, column, both diagonals) and count how far the same player's marks extend in both directions. With a target of 3 that is a handful of cells per move, instead of rescanning the whole board.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalised to k players, an n x n board and a fixed run length of 3.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def status(player):
    for r, c in all_cells(board):
        for dr, dc in DIRECTIONS:
            cells = line_of_3(r, c, dr, dc)
            if all(board[x][y] == player
                   for x, y in cells):
                return "WIN"
    if is_full(board):
        return "DRAW"
    return "PLAYING"
```

TIME	SPACE
O(n^2) per move	O(n^2)

WHY IT IS SLOW

Scans O(n^2) cells per move. Only lines through the cell just played can have changed, so checking four directions around it is O(1).

OPTIMAL SOLUTION · Check the four lines through the new mark

TIME	SPACE
O(1) per move (O(target) when generalised)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes when the winning length is a parameter w?

Replace the constant 3 by w in the run check: the cost per move becomes O(w) because each of the four directions walks at most w - 1 cells each way. The board size does not matter any more, which is the point of checking locally.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Tic-Tac-Toe System with K Players and
# 3-in-a-Row Win
#
# Design a Tic-Tac-Toe system for k players on an n x n
# board. After each move, return the game status; any
# player wins immediately after forming 3 consecutive
# marks in a straight line. Input parameters: k players
# and an n x n board. Players take turns placing their
# marks on empty cells. A player wins as soon as they form
# 3 consecutive marks in a straight line, regardless of
# how large n is. After each move, return the correct game
# status, such as Game Over or <player> won. Discuss
# implementation details and time complexity before
```

```
# coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes when the winning length is a parameter
# w?
#
# INTERVIEW NOTES
# A move can only create a win through the cell just
# played, so after each move look only along the four
# lines through that cell (row, column, both diagonals)
# and count how far the same player's marks extend in both
# directions. With a target of 3 that is a handful of
# cells per move, instead of rescanning the whole board.
#
# BRUTE FORCE (say it first): Rescan the whole board after
# every move
# After each move scan every cell and every direction for
# a run of 3. Correct and a good oracle; wasteful, because
# only lines through the new mark can have changed. Time
# O(n^2) per move. Space O(n^2).


# HELPERS for the brute force
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BRUTE FORCE (say it first): rescan the whole board
class TicTacToeBrute:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
```

```
        self.over = False

    def winner(self):
        # every cell, every direction: is there a run?
        n, t, b = self.n, self.target, self.board
        for i in range(n):
            for j in range(n):
                for di, dj in DIRECTIONS:
                    cells = [(i + di * s, j + dj * s)
                             for s in range(t)]
                    inside = all(0 <= x < n and 0 <= y < n
                                 for x, y in cells)
                    if inside and b[i][j] and all(
                            b[x][y] == b[i][j]
                            for x, y in cells):
                        return b[i][j]
        return 0

    def move(self, player, r, c):
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not (0 <= r < self.n and 0 <= c < self.n):
            raise ValueError("illegal cell")
        if self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        if self.winner():
            self.over = True
            return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"


# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
```

```
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately.
# FOLLOW-UP 1: What changes when the winning length is a
# parameter w?
```

```
# Answer: Replace the constant 3 by w in the run check:
# the cost per move becomes O(w) because each of the four
# directions walks at most w - 1 cells each way. The board
# size does not matter any more, which is the point of
# checking locally.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        # marks of the same player in a line through (r,
        # c)
        player, run = self.board[r][c], 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.inside(a, b) and \
                    self.board[a][b] == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        # whose turn is it? players go 1, 2, ..., k, 1,
        # ...
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        # only lines through the new mark can have changed
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.over = True
                return f"player {player} wins"
        if self.moves == self.n * self.n:
```

```
            self.over = True
            return "draw"
        return "continue"


if __name__ == "__main__":
    g = TicTacToe(3, 2)
    plan = [(1, 0, 0), (2, 1, 0), (1, 0, 1),
            (2, 1, 1), (1, 0, 2)]
    results = [g.move(p, r, c) for p, r, c in plan]
    assert results[:-1] == ["continue"] * 4
    assert results[-1] == "player 1 wins"
    # three players on a 4 x 4 board: player 2 wins down
    h = TicTacToe(4, 3)
    seq = [(1, 0, 0), (2, 0, 3), (3, 3, 3), (1, 1, 0),
           (2, 1, 3), (3, 3, 0), (1, 2, 1), (2, 2, 3)]
    out = [h.move(p, r, c) for p, r, c in seq]
    assert out[-1] == "player 2 wins"
    # wrong turn and used cell are rejected
    for bad in ((2, 0, 0), (1, 5, 5)):
        try:
```

```
            TicTacToe(3, 2).move(*bad)
            raise AssertionError(bad)
        except ValueError:
            pass
    # the win length is a parameter: 4 in a row
    w = TicTacToe(5, 2, target=4)
    for p, r, c in [(1, 0, 0), (2, 1, 0), (1, 0, 1),
                    (2, 1, 1), (1, 0, 2), (2, 1, 2)]:
        assert w.move(p, r, c) == "continue"
    # the brute force agrees with the fast version
    import random
    for _ in range(100):
        fast, slow = TicTacToe(4, 3), TicTacToeBrute(4, 3)
        cells = [(r, c) for r in range(4)
                 for c in range(4)]
        random.shuffle(cells)
        for i, (r, c) in enumerate(cells):
            p = i % 3 + 1
            got = fast.move(p, r, c)
            assert got == slow.move(p, r, c)
            if got != "continue":
                break
```

NOTES FROM CANDIDATES

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.
Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

PREPARATION

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY HERE: topic 10, Grids and Simulation

Site tags: simulation, algorithm, medium

MEDIUM | 1Point3Acres | onsite-coding | 2 of 6

Gate Entry / Exit Time Resolution with Tie-Break Rules

ASKED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 27.4** **30 DAYS 0** **PRIORITY 131/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2534 Time Taken to Cross the Door

Frequency: Single report | Last asked: 2026-05-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gate-entry-exit-scheduler

QUESTION

Onsite coding round 2: given an array indexed by person id, each containing **`(timestamp, action in {enter, exit})`**, return per-person the actual time they passed through the gate after applying tie-break rules between simultaneous enters and exits.SWEsimulationalgorithmmediumFrequencySingle reportLast asked2026-05-04Stageonsite-coding

APPROACH

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact tie-break rules in the four cases (previous second unused, entered, exited, and which wins), whether one person passes per second, and how ties between equal arrival times break (by id).

BRUTE FORCE TO SAY OUT LOUD

Simulate every second between the first and last timestamp. Works, but its cost grows with the size of the timestamps.

OPTIMAL IDEA

Sort people by (time, id) and keep two queues. At each step apply the previous-state rule to choose a queue. When both queues are empty jump to the next arrival time and reset the previous state to idle if at least one second was skipped. Linear after the sort.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Same-time enter and exit, an idle gap of exactly one second and of many seconds, all enters, all exits.

LIKELY FOLLOW-UPS

Several gates, priority classes, or answering online.

PITFALLS

Forgetting to reset the state across an idle gap. Letting the clock go backward.

HOW TO THINK ABOUT IT

Write the rule as a small state machine first: the state is what the gate did in the previous second (entered, exited or idle), and the rule picks who goes next from that state. Then replace the second-by-second loop with a loop over people: when nobody is waiting jump the clock forward, and remember that an idle gap resets the state to idle.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumption: one person passes per second; if an enter and an exit compete, the door's previous-second use decides (enter if it was an enter, otherwise exit, and exit if the door was idle).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        queue = choose(prev, enter_q, exit_q)
        if queue:
            person = queue.popleft()
            passed[person] = t
            prev = name_of(queue)
        t += 1
    return passed
```

TIME	SPACE
O(T + n) where T is the last timestamp	**O(n)**

WHY IT IS SLOW

It runs through every second between the first and last timestamp, so a timestamp of 10^9 means a billion iterations. Jumping to the next arrival when the queues are empty makes it O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n) for the sort, O(n) simulation	**O(n)**

OPTIMAL SOLUTION · Event-driven simulation with two queues

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

FOLLOW-UP 2 What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

FOLLOW-UP 3 Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

FOLLOW-UP 4 The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Gate Entry / Exit Time Resolution with
# Tie-Break Rules
#
# Onsite coding round 2: given an array indexed by person
# id, each containing (timestamp, action in {enter, exit})
,
# return per-person the actual time they passed through
# the gate after applying tie-break rules between
# simultaneous enters and exits.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the optimization need care across an empty
# gap?
# 2. What is the complexity, and what if events come
# online?
# 3. Timestamps go up to 10^9.
# 4. The tie-break when the gate was idle differs in this
# version of the question.
#
# INTERVIEW NOTES
# Write the rule as a small state machine first: the state
# is what the gate did in the previous second (entered,
# exited or idle), and the rule picks who goes next from
# that state. Then replace the second-by-second loop with
# a loop over people: when nobody is waiting jump the
# clock forward, and remember that an idle gap resets the
# state to idle.
#
# BRUTE FORCE (say it first): Simulate every second
# Advance time one second at a time, add arrivals to two
# queues, and pick a queue by the previous-state rule.
# Easy to reason about and the oracle for the event-driven
# version, but it spends time on every empty second. Time
# O(T + n) where T is the last timestamp. Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Simulate every second | time
# O(T + n) where T is the last timestamp
def gate_ticks(events):
    n = len(events)
    order = sorted(
        range(n), key=lambda i: (events[i][0], i)
    )
    res, ent, ext, prev, t, k = (
        [None] * n,
        deque(),
        deque(),
        None,
        0,
        0,
    )
    while any(r is None for r in res):
        while k < n and events[order[k]][0] <= t:
            (
                ent
                if events[order[k]][1] == "enter"
                else ext
            ).append(order[k])
            k += 1
        if ent and ext:
            q = ent if prev == "enter" else ext
        else:
            q = ent or ext or None
        if q is None:
            prev = None
        else:
            p = q.popleft()
            res[p], prev = t, events[p][1]
        t += 1
    return res


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state.
# FOLLOW-UP 4: The tie-break when the gate was idle
# differs in this version of the question.
# Answer: Keep the simulation and make the idle winner a
# parameter: only the line that picks the queue changes.
# With 'exit first' it equals the standard rule; with
# 'enter first' an enter and an exit arriving together
# swap places.
```

```
def gate_pass_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit")
    n = len(events)
```

```
    # people sorted by arrival, then by id
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    queues = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or queues["enter"] or queues["exit"]:
        if not queues["enter"] and not queues["exit"]:
            # nobody waits: jump the clock to the next
            # arrival
            t = max(t, events[order[i]][0])
            # a gap of idle time forgets the previous
            # action
            if t > last_used + 1:
                prev = None
        # everyone who arrived by now joins their line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            queues[events[person][1]].append(person)
            i += 1
        both = queues["enter"] and queues["exit"]
        if both:
            # FOLLOW-UP 4: idle tie-break set by
            # idle_first
            # tie: continue the direction used last second
            # (when idle, use idle_first)
            want = prev or idle_first
        else:
            want = "enter" if queues["enter"] else "exit"
        person = queues[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result


if __name__ == "__main__":
    # one enter and one exit arrive together: exit first
    ev = [(0, "enter"), (0, "exit")]
    assert gate_pass_times(ev) == [1, 0]
    # with "enter" winning the idle tie it swaps
    assert gate_pass_times(ev, "enter") == [0, 1]
    # the same direction continues while people keep
    # coming
    ev2 = [(0, "enter"), (0, "enter"), (1, "exit")]
```

```
    assert gate_pass_times(ev2) == [0, 1, 2]
    # a long idle gap resets the state
    ev3 = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert gate_pass_times(ev3) == [0, 11, 10]
    # huge timestamps cost nothing extra
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert gate_pass_times(far) == [10 ** 9 + 1, 10 ** 9]
```

NOTES FROM CANDIDATES

The clean implementation uses two queues (pending enters, pending exits) plus a prev_state variable; at each tick peek both queues, apply the four rules, advance, and update prev_state.
A naive tick-by-tick simulation walks every integer time unit from min to max timestamp; the expected optimization is to jump directly to the next event timestamp in the input, dropping complexity to O(n).
The optimization has to carry the "prior moment" state across empty gaps correctly (a 100-tick gap still counts as prev_state = none).
Walk through one or two examples on the whiteboard before coding — interviewers in this family prefer candidates who fully verbalize the precedence rules before writing.

PREPARATION

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

STUDY HERE: topic 10, Grids and Simulation

HARD | IN APP LC 2536 | PracHub | Technical Screen | 3 of 6

Count Overlapping Rectangle Updates on a Grid

POSTED 2026-07-10 | 102 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 18.1** **30 DAYS 0** **PRIORITY 151/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2536 Increment Submatrices by One

LeetCode match: LC 2536 Increment Submatrices by One

Posted on PracHub: 2026-07-10 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/count-overlapping-rectangle-updates-on-a-grid

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement `range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

APPROACH

2D difference array: for each rectangle do +1 at (r1,c1), -1 at (r1,c2+1) and (r2+1,c1), +1 at (r2+1,c2+1); then take 2D prefix sums once. O(n^2 + q) instead of O(n^2*q).

HOW TO THINK ABOUT IT

Adding 1 to every cell of up to 100,000 rectangles one cell at a time is too slow. Record only the corners of each rectangle (+1 at the top-left, -1 just beyond the right and bottom edges, +1 at the bottom-right corner beyond), and rebuild the counts with one pass of 2D prefix sums. Each rectangle then costs O(1) instead of its area.

Basis: LeetCode 2536, Increment Submatrices by One (2D difference array).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def range_add_counts(n, rectangles):
    grid = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rectangles:
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                grid[r][c] += 1
    return grid
```

TIME	SPACE
O(R * area)	O(n^2)

WHY IT IS SLOW

O(R * area): 100,000 rectangles of area up to 250,000 is billions of cell updates. A 2D difference array touches four corners per rectangle and one prefix-sum pass rebuilds the counts, O(R + n^2).

OPTIMAL SOLUTION · 2D difference array and prefix sums

TIME	SPACE
O(R + n^2)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + n^2).

FOLLOW-UP 2 The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. **Cost:** O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

FOLLOW-UP 3 Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Overlapping Rectangle Updates on a
# Grid
#
# # Count Overlapping Rectangle Updates on a Grid
# Implement range_add_counts(n, rectangles). Start with an
# n x n matrix of zeroes. Each rectangle is represented as
# [row1, col1, row2, col2] with inclusive, zero-based
# coordinates. For every rectangle, add one to each matrix
# cell inside that rectangle. Return the final matrix,
# where each cell therefore contains the number of input
# rectangles covering it. ## Example For n = 3 and
# rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]], return: text
# [ [1, 1, 0], [1, 2, 1], [0, 1, 1] ] ## Constraints - 1
# <= n <= 500 - 0 <= len(rectangles) <= 100_000 - 0 <=
# row1 <= row2 < n - 0 <= col1 <= col2 < n
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many cells are covered by at least k rectangles?
# 2. The grid is huge (10^9 x 10^9) but there are few
# rectangles.
# 3. Rectangles are added and the count of a cell is
# queried online.
#
# INTERVIEW NOTES
# Adding 1 to every cell of up to 100,000 rectangles one
# cell at a time is too slow. Record only the corners of
# each rectangle (+1 at the top-left, -1 just beyond the
# right and bottom edges, +1 at the bottom-right corner
# beyond), and rebuild the counts with one pass of 2D
# prefix sums. Each rectangle then costs O(1) instead of
# its area.
#
# BRUTE FORCE (say it first): Add one to every cell of
# every rectangle
# Loop over each rectangle and increment all of its cells.
# Correct and the oracle; with 100,000 rectangles of area
# up to 250,000 it is far too slow. Time O(R * area).
# Space O(n^2).


# BRUTE FORCE (say it first): Add one to every cell of
# every rectangle | time O(R * area)
def range_add_brute(n, rects):
    g = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rects:
```

```
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                g[r][c] += 1
    return g


# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
# in an (n + 1) x (n + 1) array.
def range_add_counts(n, rects):
    # rects are [r1, c1, r2, c2], inclusive. Mark only the
    # 4
    # corners of each rectangle in a difference array ...
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    # ... then two prefix sums spread them over the cells
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    return [row[:n] for row in d[:n]]


# FOLLOW-UP 1: How many cells are covered by at least k
# rectangles?
# Answer: Build the count matrix with the difference array
# and count the cells with value at least k. O(R + n^2).
def cells_at_least(n, rects, k):
    # how many cells are covered by k or more rectangles?
    counts = range_add_counts(n, rects)
    return sum(1 for row in counts for v in row if v >= k)


# FOLLOW-UP 2: The grid is huge (10^9 x 10^9) but there
# are few rectangles.
# Answer: Coordinate-compress the corner coordinates and
# apply the same difference array on the compressed grid
```

```
# (at most (2R)^2 cells), or use a sweep line with a
# segment tree over columns.
def cells_at_least_compressed(rects, k):
    # a huge grid, few rectangles: compress the
    # coordinates
    rows = sorted({r for r1, _, r2, _ in rects
                   for r in (r1, r2 + 1)})
    cols = sorted({c for _, c1, _, c2 in rects
                   for c in (c1, c2 + 1)})
    ri = {r: i for i, r in enumerate(rows)}
    ci = {c: i for i, c in enumerate(cols)}
    shape = (len(rows) + 1, len(cols) + 1)
    d = [[0] * shape[1] for _ in range(shape[0])]
    for r1, c1, r2, c2 in rects:
        a, b = ri[r1], ci[c1]
        e, f = ri[r2 + 1], ci[c2 + 1]
        d[a][b] += 1
        d[a][f] -= 1
        d[e][b] -= 1
        d[e][f] += 1
    total = 0
    run_col = [0] * (len(cols) + 1)
    for i in range(len(rows) - 1):
        row, run = [0] * (len(cols) + 1), 0
        for j in range(len(cols) - 1):
            run_col[j] += d[i][j]
            run += run_col[j]
            # each compressed cell stands for a block of
            # cells
            if run >= k:
                total += (rows[i + 1] - rows[i]) * (
                    cols[j + 1] - cols[j])
    return total


# FOLLOW-UP 3: Rectangles are added and the count of a
# cell is queried online.
# Answer: Use a 2D Fenwick tree with range update and
# point query (the difference-array idea turned into a
# BIT): O(log^2 n) per operation.
class Fenwick2D:
    """Online updates and point reads: O(log^2 n)."""

    def __init__(self, n):
        self.n = n
```

```
        self.t = [[0] * (n + 2) for _ in range(n + 2)]

    def _add(self, r, c, v):
        i = r + 1
        while i <= self.n + 1:
            j = c + 1
            while j <= self.n + 1:
                self.t[i][j] += v
                j += j & -j
```

```
            i += i & -i

    def add_rect(self, r1, c1, r2, c2):
        corners = ((r1, c1, 1), (r1, c2 + 1, -1),
                   (r2 + 1, c1, -1), (r2 + 1, c2 + 1, 1))
        for r, c, v in corners:
            self._add(r, c, v)

    def get(self, r, c):
        total, i = 0, r + 1
        while i > 0:
            j = c + 1
            while j > 0:
                total += self.t[i][j]
                j -= j & -j
            i -= i & -i
        return total


if __name__ == "__main__":
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    assert range_add_counts(3, rects) == [
        [1, 1, 0], [1, 2, 1], [0, 1, 1]]
    assert cells_at_least(3, rects, 2) == 1
    assert cells_at_least(3, rects, 1) == 7
    # the compressed version gives the same area
    assert cells_at_least_compressed(rects, 2) == 1
    assert cells_at_least_compressed(rects, 1) == 7
    big = [[0, 0, 10 ** 9, 10 ** 9], [5, 5, 10, 10]]
    assert cells_at_least_compressed(big, 2) == 36
    # online updates and point queries
    f = Fenwick2D(3)
    for r in rects:
        f.add_rect(*r)
    assert f.get(1, 1) == 2 and f.get(0, 2) == 0
```

FUNCTION AND TEST CASES

Signature

```
def range_add_counts(n: int, rectangles: list[list[int]])
-> list[list[int]]:
```

Sample call

```
range_add_counts(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[[0]]`
`(1, [[0, 0, 0, 0]])`	`[[1]]`

STUDY HERE: topic 10, Grids and Simulation

HARD | IN APP LC 289 | PracHub | Technical Screen | 4 of 6

Simulate In-Place Cellular State Updates

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 23.4 30 DAYS 0 PRIORITY 143/228

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #289 Game of Life

LeetCode match: LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-07)

Original: https://prachub.com/coding-questions/simulate-in-place-cellular-state-updates

QUESTION

Overview: This question evaluates a candidate's ability to implement grid-based state transitions and manage in-place updates with strict space constraints, assessing skills in matrix manipulation, neighbor traversal, and correctness under simultaneous update rules.

APPROACH

Encode old and new state together in each cell (extra bits or special values like 2 and 3) so neighbors still read the old state, then finalize in a second pass. O(mn) time, O(1) extra space.

HOW TO THINK ABOUT IT

'Update every cell at once, in place' means a cell's new value must not disturb its neighbors' reads. The trick is to store both the old and the new state in one cell (two bits), then decode in a second pass. Whenever you need old and new values together with no extra grid, think 'pack two states into one number'.

Basis: LeetCode 289, Game of Life.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def step(board):
    old = [row[:] for row in board]
    for r, c in all_cells(old):
        live = live_neighbours(old, r, c)
        alive = old[r][c] == 1
        if alive and live in (2, 3):
            board[r][c] = 1
        elif not alive and live == 3:
            board[r][c] = 1
        else:
            board[r][c] = 0
```

BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Time O(m * n) is already optimal; the cost is a second full board, O(m * n) extra space. Storing the next state in a spare bit of each cell removes the copy.

OPTIMAL SOLUTION · Two-bit encoding in the same board

TIME	SPACE
O(m * n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

FOLLOW-UP 2 Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

FOLLOW-UP 3 Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Simulate In-Place Cellular State Updates
#
# This question evaluates a candidate's ability to
# implement grid-based state transitions and manage in-
# place updates with strict space constraints, assessing
# skills in matrix manipulation, neighbor traversal, and
# correctness under simultaneous update rules.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The board is infinite, or huge and mostly empty.
# 2. Why is bit 0 used when counting neighbors?
# 3. Cells wrap around the edges (a torus).
#
# INTERVIEW NOTES
# 'Update every cell at once, in place' means a cell's new
```

```
# value must not disturb its neighbors' reads. The trick
# is to store both the old and the new state in one cell
# (two bits), then decode in a second pass. Whenever you
# need old and new values together with no extra grid,
# think 'pack two states into one number'.
#
# BRUTE FORCE (say it first): Copy the board, read from
# the copy
# Compute the next generation from an untouched copy of
# the board. It is the simplest correct answer, and the
# extra copy is exactly what the in-place version removes.
# Time O(m * n). Space O(m * n).


from collections import Counter


# BRUTE FORCE (say it first): Copy the board, read from
# the copy | time O(m * n)
def _life_neighbors(board, r, c, live):
    return sum(
        live(board[nr][nc])
        for nr in range(r - 1, r + 2)
        for nc in range(c - 1, c + 2)
        if (nr, nc) != (r, c)
        and 0 <= nr < len(board)
        and 0 <= nc < len(board[0])
    )
```

```
def life_brute(board):
    old = [row[:] for row in board]
    for r in range(len(board)):
        for c in range(len(board[0])):
            n = _life_neighbors(
                old, r, c, lambda v: v == 1
            )
            board[r][c] = (
                1
                if (old[r][c] == 1 and n in (2, 3))
                or (old[r][c] == 0 and n == 3)
                else 0
            )
    return board


# HELPERS for the base question
AROUND = [(dr, dc) for dr in (-1, 0, 1)
          for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]


def live_neighbors(board, r, c, wrap=False):
    m, n = len(board), len(board[0])
    count = 0
    for dr, dc in AROUND:
        a, b = r + dr, c + dc
        if wrap:
            # a torus: the edges are glued together
            a, b = a % m, b % n
        elif not (0 <= a < m and 0 <= b < n):
            continue
        # bit 0 is the OLD state, even for updated cells
        count += board[a][b] & 1
    return count


# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules.
def step_in_place(board, wrap=False):
    m, n = len(board), len(board[0])
```

```
    for r in range(m):
        for c in range(n):
```

```
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            # bit 1 stores the NEW state
            if live == 3 or (alive and live == 2):
                board[r][c] |= 2
    for r in range(m):
        for c in range(n):
            # shift the new state into place
            board[r][c] >>= 1
    return board


# FOLLOW-UP 1: The board is infinite, or huge and mostly
# empty.
# Answer: Store only the live cells in a set. A cell's
# next state depends on its live neighbor count, so count
# how many live neighbors every candidate cell has with a
# Counter and keep cells with 3, or with 2 if already
# live.
def step_sparse(live):
    # infinite or huge board: only the live cells are
    # stored
    counts = Counter()
    for r, c in live:
        for dr, dc in AROUND:
            counts[(r + dr, c + dc)] += 1
    return {cell for cell, k in counts.items()
            if k == 3 or (k == 2 and cell in live)}


if __name__ == "__main__":
    board = [[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]]
    want = [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
    assert step_in_place(board) == want
    # a blinker flips between a row and a column
    blinker = {(1, 0), (1, 1), (1, 2)}
    flipped = {(0, 1), (1, 1), (2, 1)}
    assert step_sparse(blinker) == flipped
    assert step_sparse(step_sparse(blinker)) == blinker
    # a block never changes
    block = {(0, 0), (0, 1), (1, 0), (1, 1)}
    assert step_sparse(block) == block
    # the sparse and dense versions agree on a glider
    glider = {(0, 1), (1, 2), (2, 0), (2, 1), (2, 2)}
    grid = [[0] * 6 for _ in range(6)]
```

```
    for r, c in glider:
        grid[r][c] = 1
    dense = step_in_place(grid)
    assert {(r, c) for r in range(6) for c in range(6)
            if dense[r][c]} == step_sparse(glider)
    # on a torus the left and right edges touch
    ring = [[1, 0, 1]]
    assert live_neighbors(ring, 0, 0, wrap=True) >= 1
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]])
```

Test cases

Input	Expected
`([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]],)`	`[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]`
`([[1, 1], [1, 1]],)`	`[[1, 1], [1, 1]]`

STUDY HERE: topic 10, Grids and Simulation

Site tags: simulation, data-structure, hard

HARD | 1Point3Acres | onsite-coding | 5 of 6

Babylon Tile-Stack Merge Game

ASKED 2025-12-29 | 295 DAYS AGO | OLDER

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 191/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-12-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

QUESTION

Onsite NG round 1 (Munich office): implement a two-player tile-stack merge game ("Babylon") given the rules. Start state is 12 height-1 stacks of 4 colors (Y / W / G / B), 3 of each; a move places one stack on top of another under a constraint; the player who cannot move loses.

12 tiles initially, partitioned into 12 height-1 stacks.
4 colors: Y / W / G / B; exactly 3 tiles per color.
Two players alternate moves.
A move = pick two stacks and place one entirely on top of the other (merge).

Legal iff at least one of: (a) the two stacks have the same height, OR (b) their top-most tiles share the same color.
After merge: new height = sum of the two heights, new top color = color of the stack placed on top.

The player who has no legal move on their turn loses.
Implement the full game so it can be played out (legal-move detection, state transition, terminal check).

EXAMPLES

Start: 12 stacks of height 1, with tops Y Y Y W W W G G G B B B. A legal opening move: stack the first Y on the second Y (same color top), producing one height-2 stack of color Y plus 10 singletons.

APPROACH

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

HOW TO THINK ABOUT IT

Model a state as the list of stacks, each as (height, top colour). A legal move puts stack i on stack j when the heights are equal or the top colours are equal; the new stack has the summed height and the colour of the stack that was placed on top. The player with no legal move loses, so a state is winning when some move leads to a losing state. The order of stacks does not matter, so sort the list to share states in a cache.

Basis: Written for this question: a two-player impartial game (the player who cannot move loses), solved by memoised search over the multiset of stacks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(position):
    for i, j in legal_moves(position):
        nxt = merge(position, i, j)
        if not wins(nxt):
            return True  # opponent loses
    return False
```

TIME	SPACE
exponential	O(depth)

WHY IT IS SLOW

Explores the same positions many times, exponential. Sorting the stacks and caching each position's result visits every distinct position once.

OPTIMAL SOLUTION · Negamax with memoisation on the sorted stack tuple

TIME	SPACE
O(states * moves^2)	O(states)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How big is the real state space?

With 12 starting stacks the first position has at most 66 unordered pairs and the number of reachable multisets of (height, colour) stacks shrinks quickly because every merge removes a stack, so memoised search is feasible. In the interview build the clean state class, legal moves and play(move) first and add the solver only if there is time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Babylon Tile-Stack Merge Game
#
```

```
# Onsite NG round 1 (Munich office): implement a two-
# player tile-stack merge game ("Babylon") given the
# rules. Start state is 12 height-1 stacks of 4 colors (Y
# / W / G / B), 3 of each; a move places one stack on top
# of another under a constraint; the player who cannot
# move loses. 12 tiles initially, partitioned into 12
# height-1 stacks. 4 colors: Y / W / G / B; exactly 3
# tiles per color. Two players alternate moves. A move =
# pick two stacks and place one entirely on top of the
# other (merge). Legal iff at least one of: (a) the two
# stacks have the same height, OR (b) their top-most tiles
# share the same color. After merge: new height = sum of
# the two heights, new top color = color of the stack
# placed on top. The player who has no legal move on their
# turn loses. Implement the full game so it can be played
# out (legal-move detection, state transition, terminal
# check).
#
# FOLLOW-UPS (answered below, in this order):
# 1. How big is the real state space?
#
# INTERVIEW NOTES
# Model a state as the list of stacks, each as (height,
# top colour). A legal move puts stack i on stack j when
# the heights are equal or the top colours are equal; the
# new stack has the summed height and the colour of the
# stack that was placed on top. The player with no legal
# move loses, so a state is winning when some move leads
# to a losing state. The order of stacks does not matter,
# so sort the list to share states in a cache.
#
# BRUTE FORCE (say it first): Plain recursion over every
# move
# Try every legal move recursively with no caching.
# Correct and fine for a few stacks; it recomputes the
# same positions over and over. Time exponential. Space
# O(depth).


import itertools


# BRUTE FORCE (say it first): Plain recursion over every
```

```
# move | time exponential
def babylon_brute(stacks):
    stacks = list(stacks)
    for i, j in itertools.permutations(
        range(len(stacks)), 2
    ):
        (h1, c1), (h2, c2) = stacks[i], stacks[j]
        if h1 == h2 or c1 == c2:
            rest = [
                s
                for k, s in enumerate(stacks)
                if k not in (i, j)
            ] + [(h1 + h2, c1)]
            if not babylon_brute(rest):
                return True
    return False


# HELPERS for the base question
class Babylon:
    # a stack is (height, colour of its top tile)
    def __init__(self, stacks=None):
        if stacks is None:
            # 12 tiles: Y, W, G, B three times each
            stacks = [(1, c) for c in "YWGB" * 3]
        self.stacks = list(stacks)

    def legal_moves(self):
        # put stack i on top of stack j
```

```
        moves = []
        for i, j in itertools.permutations(
                range(len(self.stacks)), 2):
            h1, c1 = self.stacks[i]
            h2, c2 = self.stacks[j]
            # same height, or the same colour on top
            if h1 == h2 or c1 == c2:
                moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move (the original is kept)
        (h1, c1), (h2, _) = self.stacks[i], self.stacks[j]
        rest = [s for k, s in enumerate(self.stacks)
                if k not in (i, j)]
        # heights add; the top colour is the moved stack's
```

```
        return rest + [(h1 + h2, c1)]

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        # the player with no move loses
        return not self.legal_moves()


# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
# true when some move reaches a state where the opponent
# loses.
def wins(stacks):
    # does the player to move win with best play?
    cache = {}

    def solve(state):
        if state in cache:
            return cache[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            nxt = tuple(sorted(game.after(i, j)))
            # a move to a position the opponent loses wins
            if not solve(nxt):
                result = True
                break
        cache[state] = result
        return result

    # sorted stacks: equal positions share one cache entry
    return solve(tuple(sorted(stacks)))


# PICK the winning move itself
def best_move(game):
    # pick a move that leaves the opponent lost, if any
    for i, j in game.legal_moves():
        if not wins(game.after(i, j)):
```

```
            return i, j
    moves = game.legal_moves()
    return moves[0] if moves else None


if __name__ == "__main__":
    # one stack: no moves, so the player to move loses
    assert Babylon([(1, "Y")]).is_over()
    assert not wins([(1, "Y")])
    # two equal stacks: one move ends the game, so we win
```

```
assert wins([(1, "Y"), (1, "W")])
g = Babylon([(1, "Y"), (1, "Y"), (1, "W")])
assert (0, 1) in g.legal_moves()
# play a whole game with the solver; the loser is
# stuck
while not g.is_over():
    g.play(*best_move(g))
assert len(g.stacks) >= 1
try:
    Babylon([(1, "Y"), (2, "W")]).play(0, 1)
    raise AssertionError("move should be illegal")
except ValueError:
    pass
# the real start has 12 stacks and up to 132 ordered
# moves
assert len(Babylon().legal_moves()) > 0
```

NOTES FROM CANDIDATES

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API.
Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".
Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

PREPARATION

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

STUDY HERE: topic 10, Grids and Simulation

HARD | PracHub | Technical Screen | 6 of 6

Count same-color squares in a character grid

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE **ALL TIME 51.2** **30 DAYS 0** **PRIORITY 66/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1277 Count Square Submatrices with All Ones, LeetCode #221 Maximal Square

LeetCode match: LC 1277 Count Square Submatrices with All Ones / LC 221 Maximal Square

Posted on PracHub: 2025-11-22 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/count-same-color-squares-in-a-character-grid

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has `m` rows and `n` columns.
- `grid[i][j]` is an alphabetic character (e.g., `'a'–'z'` or `'A'–'Z'`). A **square** in this grid is defined as a contiguous `k × k` submatrix aligned with the grid axes, for some integer `k >= 1`. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by `'a'`. Your function should return the total count of such monochromatic squares for a given input grid.

QUESTION

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

APPROACH

dp[i][j] = 1 + min(top, left, top-left) when those neighbors share the same char as (i,j), else 1. Sum all dp values = number of monochromatic squares. O(mn).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the grid size, whether every k x k square counts (k from 1 up), and whether to count squares of every size (the statement says all).

BRUTE FORCE TO SAY OUT LOUD

For every top-left corner and size, check all cells. O(mn * min(m, n)^3).

OPTIMAL IDEA

dp[i][j] is the size of the largest same-color square whose bottom-right corner is (i, j), and it equals the number of such squares ending there. If i and j are positive and the cells above, left and diagonal match the current character, dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]); otherwise it is 1. The answer is the sum of all dp values. O(mn) time, O(n) with a rolling row.

DRY-RUN THESE CASES

A 1 x 1 grid (1), a grid of one color (sum of squares counts), a checkerboard (only 1 x 1 squares), a rectangle grid.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The largest square only, or squares allowing at most one different cell.

PITFALLS

Forgetting to compare the three neighbors to the current character, not just to each other.

HOW TO THINK ABOUT IT

Let dp[i][j] be the side of the largest same-colour square whose bottom-right corner is (i, j). That is also the number of squares ending at that corner (one per size from 1 up to it). A square of side k at (i, j) needs squares of side k - 1 at the three cells above, left and above-left, all of the same colour, hence the min of those three plus one.

Basis: LeetCode 1277 (Count Square Submatrices with All Ones) with 'same character' instead of 1. Counts squares of every size k >= 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

The same cells are rechecked for every size: O(m * n * min(m, n)^3). A table of the largest same-colour square ending at each cell, summed over all cells, is O(m * n).

OPTIMAL SOLUTION · DP: side of the largest square ending at each cell, summed

TIME	SPACE
O(m * n)	O(n) with a rolling row

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

FOLLOW-UP 2 Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

FOLLOW-UP 3 Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count same-color squares in a character
# grid
#
# You are given a 2D grid (matrix) of characters. Each
# character represents a color: cells with the same
# character are considered the same color. Formally: - The
# grid has m rows and n columns. - grid[i][j] is an
# alphabetic character (e.g., 'a'–'z' or 'A'–'Z'). A
# **square** in this grid is defined as a contiguous k × k
# submatrix aligned with the grid axes, for some integer k
# >= 1. A square is **monochromatic** if **all** of its
# cells contain the **same** character (i.e., the same
# color). Task: - Count and return the total number of
# monochromatic squares in the grid. - Count all sizes of
# squares (1×1, 2×2, ..., up to the largest possible that
# fits in the grid). Example (just for clarity, not
# necessarily exhaustive): If the grid is: a a b a a b b b
# b Then some monochromatic squares include: - All 1×1
# cells (each single cell is a monochromatic square).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the largest square (LeetCode 221).
# 2. Allow at most one different cell inside a square.
# 3. Count only squares of colour c, or only side >= 2.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That is also
# the number of squares ending at that corner (one per
# size from 1 up to it). A square of side k at (i, j)
# needs squares of side k - 1 at the three cells above,
# left and above-left, all of the same colour, hence the
# min of those three plus one.
#
# BRUTE FORCE (say it first): Check every square from
# every corner
# For each top-left corner and each size k, verify that
# all k * k cells are equal. Correct and the oracle; the
# cost is huge because the same cells are rechecked for
# every size. Time O(m * n * min(m, n)^3). Space O(1).


# BRUTE FORCE (say it first): Check every square from
# every corner | time O(m * n * min(m, n)^3)
def same_color_squares_brute(grid):
```

```
    m, n, total = len(grid), len(grid[0]), 0
    for i in range(m):
```

```
        for j in range(n):
            for k in range(1, min(m - i, n - j) + 1):
                if all(
                    grid[x][y] == grid[i][j]
                    for x in range(i, i + k)
                    for y in range(j, j + k)
                ):
                    total += 1
                else:
                    break
    return total


# HELPERS for the base question
def square_table(grid):
    # side[i][j]: the biggest same-colour square with its
    # bottom-right corner at (i, j)
    prev, rows = [0] * len(grid[0]), []
    for i, row in enumerate(grid):
        cur = [1] * len(row)
        for j, ch in enumerate(row):
            # the up, left and up-left cells must match
            # too
            if i and j and ch == grid[i - 1][j] \
                    and ch == grid[i][j - 1] \
                    and ch == grid[i - 1][j - 1]:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        rows.append(cur)
        prev = cur
    return rows


# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    # each cell adds one square for every size up to its
    # side
    return sum(sum(row) for row in square_table(grid))
```

```
# FOLLOW-UP 1: Only the largest square (LeetCode 221).
# Answer: Keep the maximum of the same dp table instead of
# the sum.
def largest_square(grid):
    # LeetCode 221 style: only the biggest side
    return max(max(row) for row in square_table(grid))


# FOLLOW-UP 3: Count only squares of colour c, or only
# side >= 2.
# Answer: Filter while summing: add dp only for cells of
# colour c, or add max(0, dp - 1) for sizes at least 2.
def count_of_color(grid, color, min_side=1):
    # squares of one colour, and/or of side at least
    # min_side
    total = 0
    for i, row in enumerate(square_table(grid)):
        for j, side in enumerate(row):
            if grid[i][j] == color:
                # sides min_side .. side are counted
                total += max(0, side - min_side + 1)
    return total


if __name__ == "__main__":
    g = ["aab", "aab", "bbb"]
    # singles 9, plus the 2 x 2 block of a's
    assert count_same_color_squares(g) == 10
    assert largest_square(g) == 2
    assert count_of_color(g, "a") == 5
    assert count_of_color(g, "b") == 5
    full = ["zzz", "zzz", "zzz"]
    # 9 + 4 + 1 squares
    assert count_same_color_squares(full) == 14
    assert count_of_color(full, "z", 2) == 5
    assert count_of_color(full, "q") == 0
    assert count_same_color_squares(["a"]) == 1
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> int:
SAMPLE INPUT
([])
```

Test cases

Input	Expected
`([])`	`0`
`(["aab", "aab", "bbb"])`	`10`

ALSO REPORTED · 1 more

Count Same-Color Squares in Matrix | 1Point3Acres | onsite-coding | Low

https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

Preparation: Write the all-1-squares DP from memory in under 8 min.

Generalize to multi-color in <2 min — only the equality check changes.